Lab Work: String in Python

1. Employee ID Validation and Analysis System

Problem Statement

A company generates employee IDs in the following format:

EMP2026ANUJ458

Tasks

Write a program to:

1. Count the number of uppercase letters.
2. Count the number of digits.
3. Extract the joining year.
4. Extract the employee name.
5. Check whether the ID follows these rules:
    - Starts with "EMP"
    - Contains exactly 4 digits for the year
    - Ends with exactly 3 digits
6. Create a list containing all digits present in the ID.
7. Find the sum of all digits present in the ID.
8. Display whether the ID is valid or invalid.

Sample Output

Employee ID: EMP2026ANUJ458

Uppercase Letters: 7
Digits: 7

Joining Year: 2026
Employee Name: ANUJ

Digit List: [2, 0, 2, 6, 4, 5, 8]
Sum of Digits: 27

ID Status: Valid

---

2. Password Strength Analyzer

Problem Statement

A user enters a password.

Python@2026!

Tasks

Write a program to determine whether the password is Strong, Medium, or Weak.

Rules:

- Minimum length 8
- Contains at least:
    - 1 uppercase letter
    - 1 lowercase letter
    - 1 digit
    - 1 special character

Additionally:

1. Count uppercase letters.
2. Count lowercase letters.
3. Count digits.
4. Count special characters.
5. Display all digits separately.
6. Display all special characters separately.

Sample Output

Password: Python@2026!

Uppercase Letters: 1
Lowercase Letters: 5
Digits: 4
Special Characters: 2

Digits Found: ['2', '0', '2', '6']
Special Characters Found: ['@', '!']

Password Strength: Strong

---

3. Chat Message Analytics

Problem Statement

A chat application stores a message:

Python is awesome and Python is easy to learn

Tasks

Write a program to:

1. Count total characters.
2. Count total words.
3. Find the longest word.
4. Find the shortest word.
5. Count how many times the word "Python" appears.

6. Create a list of words having more than 4 characters.
7. Display all words starting with a vowel.
8. Count the number of vowels and consonants.

Sample Output

Message:
Python is awesome and Python is easy to learn

Total Characters: 45
Total Words: 8

Longest Word: awesome
Shortest Word: is

Occurrences of Python: 2

Words Longer Than 4 Characters:
['Python', 'awesome', 'Python', 'learn']

Vowels: 16
Consonants: 22

---

4. Vehicle Number Plate Verification

Problem Statement

A vehicle number plate is entered:

MH12AB4589

Tasks

Write a program to:

1. Extract state code.
2. Extract district code.
3. Extract vehicle series.
4. Extract vehicle number.
5. Count letters and digits separately.
6. Verify:
    - First 2 characters must be alphabets.
    - Next 2 must be digits.
    - Next 2 must be alphabets.
    - Last 4 must be digits.
7. Display whether the number plate is valid.

Sample Output

Vehicle Number: MH12AB4589
State Code: MH
District Code: 12
Series: AB
Vehicle Number: 4589

Total Letters: 4
Total Digits: 6

Vehicle Number Status: Valid

---

5. Product Review Analyzer

Problem Statement

A customer submits a review:

This product is excellent excellent excellent and very useful

Tasks

Write a program to:

1. Count total words.
2. Create a dictionary containing word frequencies.
3. Find the most frequently used word.
4. Find all words appearing only once.
5. Count words having more than 5 characters.
6. Display words in reverse order.
7. Create a list of unique words.

Sample Output

Total Words: 8

Word Frequencies:
This -> 1
product -> 1
is -> 1
excellent -> 3
and -> 1
very -> 1
useful -> 1

Most Frequent Word: excellent

Words Appearing Once:
['This', 'product', 'is', 'and', 'very', 'useful']

Unique Words:
['This', 'product', 'is', 'excellent', 'and', 'very', 'useful']

---

6. Email Address Validator

Problem Statement

A user enters an email address:

rahul.sharma2026@gmail.com

Tasks

Write a program to:

1. Extract username.
2. Extract domain name.
3. Extract extension.
4. Count digits present in username.
5. Count special characters.
6. Check whether:
    - Exactly one '@' exists.
    - At least one '.' exists after '@'.
7. Display Valid Email or Invalid Email.

Sample Output

Email: rahul.sharma2026@gmail.com

Username: rahul.sharma2026
Domain: gmail
Extension: com

Digits Found: 4
Special Characters Found: 2

Email Status: Valid

---

7. Username Generator System

Problem Statement

A student enters:

Rahul Sharma

Tasks

Generate a username using the rules:

1. Remove spaces.
2. Convert to lowercase.

3. Append current year (2026).
4. If username length exceeds 12, keep only first 12 characters.
5. Count vowels in the generated username.
6. Count consonants.
7. Display username statistics.

Sample Output

Original Name: Rahul Sharma

Generated Username:
rahulsharma2026

Username Length: 15

Vowels: 5
Consonants: 10

Status: Username Generated Successfully

---

8. String-Based Attendance Tracker

Problem Statement

Attendance of a student for 15 days is represented as:

PPAPPPAAPPPPAPP

Where:

- P = Present
- A = Absent

Tasks

Write a program to:

1. Count Present and Absent days.
2. Calculate attendance percentage.
3. Find the longest consecutive streak of Presence.
4. Find the longest consecutive streak of Absence.
5. Determine whether attendance is below 75%.

Sample Output

Attendance Record:
PPAPPPAAPPPPAPP

Present Days: 11
Absent Days: 4

Attendance Percentage: 73.33%

Longest Present Streak: 4
Longest Absent Streak: 2

Attendance Status: Below 75%

---

9. License Key Verification System

Problem Statement

A software license key is entered:

ABCD-EFGH-IJKL-MNOP

Tasks

Write a program to:

1. Verify there are exactly 4 groups.
2. Verify each group contains exactly 4 characters.
3. Count total letters.
4. Count vowels.
5. Remove hyphens and display the merged key.
6. Create a list containing all groups.
7. Display whether the key format is valid.

Sample Output

License Key:
ABCD-EFGH-IJKL-MNOP

Groups:
['ABCD', 'EFGH', 'IJKL', 'MNOP']

Number of Groups: 4

Total Letters: 16
Total Vowels: 4

Merged Key:
ABCDEFGHIJKLMNOP

License Key Status: Valid

---

10. Text Compression Analyzer

Problem Statement

A compressed message is given:

AAABBBCCCDDDAAA

Tasks

Write a program to:

1. Count occurrences of each character.
2. Create a dictionary of character frequencies.
3. Display unique characters.
4. Find the most frequent character.
5. Create a compressed output:

   A3B3C3D3A3
6. Calculate compression ratio.

Sample Output

Original Text:
AAABBBCCCDDDAAA

Character Frequencies:
A -> 6
B -> 3
C -> 3
D -> 3

Unique Characters:
['A', 'B', 'C', 'D']

Most Frequent Character: A

Compressed Output:
A3B3C3D3A3

Original Length: 15
Compressed Length: 10

Compression Ratio: 66.67%